Sardar Vallabhbhai National Institute of Technology, Surat

Department of Artificial Intelligence

2D Planar Plotter

Bridging Digital Image Processing with Parallel Kinematics

Course: Robotics & Its Applications

Submitted by:

Pooja Sharma	U23CS137
Sadhana Devarajan	U23AI003
Shashank Vijayvargia	U23EE025
Tanvi Patange	U23CS001
Teja Swaroop	U23AI109

SVNIT Surat • 2026

Abstract

This report presents the design, simulation, and software implementation of a **2D Planar Drawing Robot** based on a five-bar parallel kinematic mechanism. The system translates arbitrary digital vector graphics — supplied as PDF files — into precise physical pen trajectories. A complete software pipeline was developed and validated in simulation: a computer-vision front-end (PyMuPDF + OpenCV) extracts shape contours from uploaded PDFs; a coordinate-mapping module scales pixel paths to the robot's physical workspace; a Python inverse-kinematics (IK) solver computes the required joint angles; and a MuJoCo-based physics simulation executes and visualises the motion. A Flask web interface provides a drag-and-drop upload UI, real-time shape preview, and one-click simulation launch. The simulation phase is fully operational and confirms the mathematical validity of the IK model and the accuracy of the CV pipeline. Physical hardware (ESP32, NEMA 17 steppers, TMC2209 drivers, SG90 servo) is planned for the next phase.

Contents

1 Introduction **4**

2 Problem Statement **4**
2.1 The Challenge of Automated Precision Drawing 4
2.2 Objective 4

3 Core Concepts **5**
3.1 Parallel Kinematics 5
3.2 Computer Vision / Image Processing 5
3.3 Inverse Kinematics 5
3.4 Embedded Systems and Simulation 5

4 Mechanism Overview **5**
4.1 Architecture 5
4.2 Design Advantages 6
4.3 Robot Workspace 6

5 Mathematics of Inverse Kinematics **6**
5.1 Derivation 6
5.2 Coordinate Transformation 6

6 System Architecture and Workflow **7**
6.1 System Architecture 7
6.2 End-to-End Pipeline 7
6.3 Component Responsibilities 8

7 Software Implementation **8**
7.1 `pdf_parser.py` — PDF Rasterisation (Sadhana) 8
7.2 `shape_extractor.py` — Contour Detection (Tanvi) 8
7.3 `coord_mapper.py` — Workspace Mapping (Pooja) 8
7.4 `pipeline.py` — Pipeline Orchestrator (Pooja) 8
7.5 `drawing_bot.xml` — MuJoCo Robot Model (Sadhana) 8
7.6 `app.py` — Flask Web Backend (Teja) 9
7.7 `sim.py` — IK Solver and Simulation Driver (Teja) 9
7.8 Web Frontend — `index.html` (Teja) 9

8 Results and Validation **9**
8.1 Web Interface 9
8.2 Flask Server Startup 10
8.3 MuJoCo Simulation Outputs 11

9 Software Stack and Dependencies **13**
9.1 Planned Hardware 13

10 Real-World Applications **13**

11 Limitations **13**

12 Future Scope and Improvements **14**

13 Conclusion **14**

1. Introduction

Precision 2D drawing and engraving systems are widely used in industrial manufacturing, PCB fabrication, textile cutting, and artistic applications. Conventional approaches rely on heavy Cartesian (X-Y gantry) machines that suffer from high inertia, or on serial robotic arms that accumulate joint errors at the end-effector. Both architectures are ill-suited for applications demanding high speed, low inertia, and sub-millimetre repeatability.

Parallel kinematic mechanisms address these limitations by grounding all actuators at the base, leaving only lightweight passive links in motion. The five-bar closed-loop linkage used in this project distributes mechanical load across both arms simultaneously, achieving high rigidity with minimal moving mass.

This project integrates three distinct engineering domains:

1. **Computer Vision** — automatic extraction of drawing paths from PDF files using PyMuPDF and OpenCV.
2. **Kinematics** — a Python-based IK solver mapping Cartesian coordinates to joint angles via the Law of Cosines.
3. **Physics Simulation** — real-time motion execution and verification inside the MuJoCo physics engine with a custom XML robot model.

The project was carried out by a team of five students at SVNIT Surat with clearly divided module responsibilities (see Table 2).

2. Problem Statement

2.1 The Challenge of Automated Precision Drawing

Standard CNC machines use rigid, heavy X-Y gantries that are bulky and exhibit high inertia, making them unsuitable for high-speed or portable applications. Serial robotic arms suffer from *cumulative positional error* and end-effector vibration because each distal joint depends on all proximal joints being accurate.

2.2 Objective

Design, model, and simulate a **low-inertia, highly rigid drawing robot** capable of:

- Accepting an arbitrary 2D shape as a PDF input.
- Automatically extracting the shape outline using computer vision.
- Converting the outline to robot joint-space trajectories via IK.
- Executing and visualising the drawing motion in a physics simulator.
- Providing a user-friendly web interface for non-expert operation.

3. Core Concepts

3.1 Parallel Kinematics

A *parallel kinematic mechanism* (PKM) employs closed-loop mechanical linkages to distribute physical loads across multiple actuated arms. All motors are grounded, so only the passive links move at speed, yielding low moving inertia, high structural stiffness, and improved positional repeatability.

3.2 Computer Vision / Image Processing

Shape contours are extracted from a rasterised PDF image using greyscale conversion, Gaussian blur, Otsu's automatic thresholding (inverted), morphological close/open operations, and OpenCV `findContours`. An optional polygon approximation returns corner-only paths, while dense sampling preserves smooth curves.

3.3 Inverse Kinematics

For the five-bar mechanism the IK problem reduces to finding the joint angles θ_1 (left motor) and θ_2 (right motor) given a desired end-effector position (x, y). The solution uses the **Law of Cosines**:

$$\cos\theta = \frac{a^2 + b^2 - c^2}{2ab} \tag{1}$$

`get_ik_radians(x, y)` implements this via `arctan2` and `arccos`, yielding a pair of joint angles for each Cartesian waypoint.

3.4 Embedded Systems and Simulation

The simulation layer uses **MuJoCo** with a custom robot model defined in `drawing_bot.xml`. Joint angle sequences are streamed to the simulator in real time, producing a physics-accurate rendering of the drawing motion.

4. Mechanism Overview

4.1 Architecture

The robot is a **five-bar parallel mechanism** with two grounded, stationary actuators driving a closed-loop linkage. Table 1 lists the geometric parameters.

Table 1: Link parameters of the five-bar mechanism.

Parameter	Symbol	Value
Primary link length	L_1	0.15 m
Secondary link length	L_2	0.27 m
Motor base separation	D	0.075 m

4.2 Design Advantages

The closed-loop design prevents the "wobble" typical of cantilevered serial arms. Because the heavy actuators remain fixed at the base, the moving arms are extremely lightweight, drastically reducing inertia. Link lengths were chosen to maximise the usable work envelope and avoid singularity zones.

4.3 Robot Workspace

The safe drawing zone derived from the arm geometry is:

$$x \in [-0.04,\ 0.04]\ \text{m}, \qquad y \in [0.32,\ 0.40]\ \text{m}$$

Shapes are scaled uniformly to fill 85% of this zone, centred, with the image y-axis inverted to match the robot's forward direction.

5. Mathematics of Inverse Kinematics

5.1 Derivation

Let the two motor pivots be located at $(-D/2, 0)$ and $(+D/2, 0)$. For each arm, the distance from its pivot to the end-effector is:

$$r_L = \sqrt{(x + D/2)^2 + y^2} \tag{2}$$

$$r_R = \sqrt{(x - D/2)^2 + y^2} \tag{3}$$

Applying the Law of Cosines (Eq. 1) to the triangle (L_1, L_2, r) gives the elbow angle. The full joint angle is:

$$\theta = \phi \pm \arccos\left(\frac{r^2 + L_1^2 - L_2^2}{2\,r\,L_1}\right) \tag{4}$$

where $\phi = \texttt{arctan2}(y,\ x \pm D/2)$ is the bearing angle and $\pm$ selects elbow-up or elbow-down configuration.

5.2 Coordinate Transformation

Pixel coordinates (p_x, p_y) are linearly mapped to robot-workspace metres (r_x, r_y) with uniform aspect-ratio preservation and Y-axis inversion:

$$r_x = c_x + (p_x - s_{cx}) \cdot k \tag{5}$$

$$r_y = c_y - (p_y - s_{cy}) \cdot k \tag{6}$$

where $k = \min(w_w/S_w,\ w_h/S_h)$ is the uniform scale factor, (c_x, c_y) is the workspace centre, and (s_{cx}, s_{cy}) is the shape centroid in pixel space.

6. System Architecture and Workflow

6.1 System Architecture

Figure 1 shows the complete system architecture.

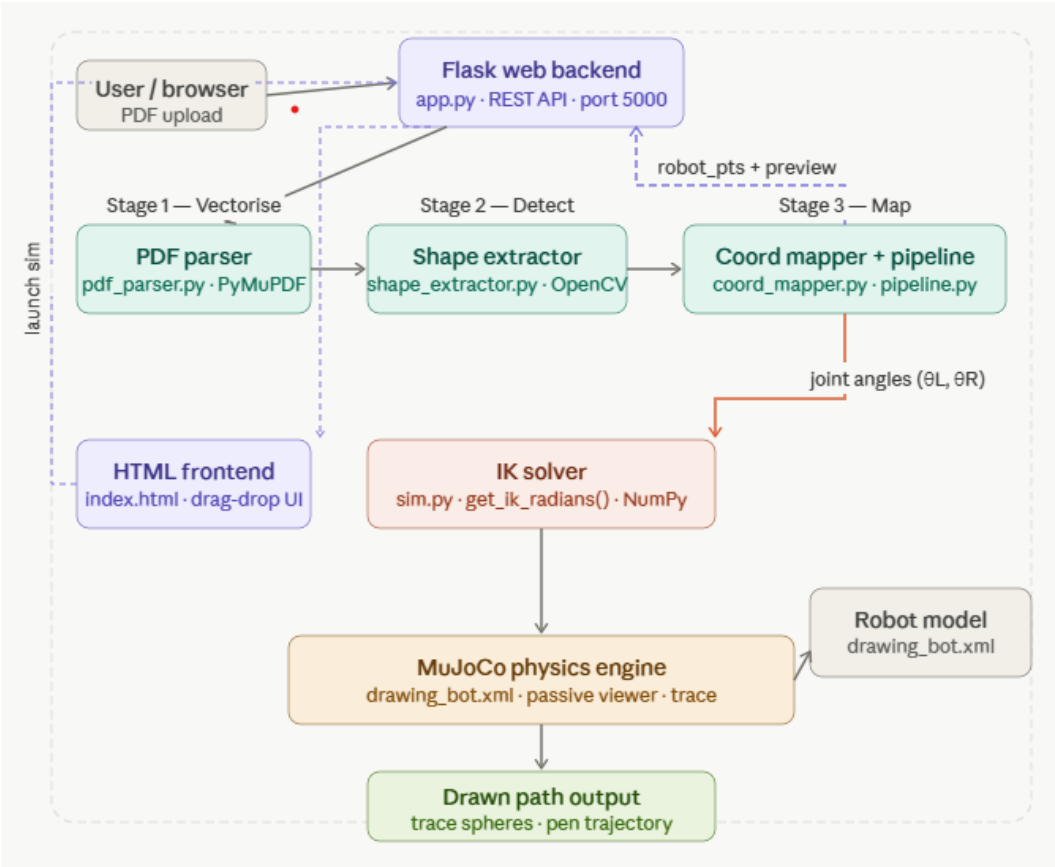

Figure 1: System architecture of the 2D Planar Plotter. The Flask backend orchestrates the CV pipeline (Stages 1–3), streams waypoints to the IK solver, and drives the MuJoCo physics simulation.

6.2 End-to-End Pipeline

Stage 1. Vectorisation — The PDF is rendered at 300 DPI via PyMuPDF. OpenCV contour detection extracts the primary shape boundary as an ordered list of pixel coordinates.

Stage 2. Path Generation — `coord_mapper.py` scales the pixel path to robot workspace metres with uniform aspect-ratio preservation and Y-axis inversion. `pipeline.py` orchestrates stages 1–3 in a single `run_pipeline()` call.

Stage 3. IK Computation — `sim.py` calls `get_ik_radians()` for each waypoint to produce joint-angle pairs $(\theta_L,\ \theta_R)$.

Stage 4. Simulation & Execution — MuJoCo executes the joint trajectory. The Flask backend allows drag-and-drop PDF upload, live preview, and one-click simulation launch.

6.3 Component Responsibilities

Table 2: Module-to-team-member responsibility matrix.

Team Member	Roll No.	Modules
Pooja Sharma	U23CS137	`coord_mapper.py`, `pipeline.py`
Sadhana	U23AI003	`pdf_parser.py`, `drawing_bot.xml`
Tanvi Patange	U23CS001	`shape_extractor.py`, `requirements.txt`
Teja Swaroop	U23AI109	`app.py`, `sim.py`
Shashank Vijayvargia	U23EE025	System integration, IK mathematics

7. Software Implementation

7.1 `pdf_parser.py` — PDF Rasterisation (Sadhana)

`pdf_parser.py` converts a PDF page into a BGR NumPy array using PyMuPDF (`fitz`), eliminating the Poppler dependency that complicates Windows installation. The function renders at 300 DPI (zoom = 300/72), producing approximately 3508×2480 pixels for an A4 page — sufficient for accurate contour detection.

7.2 `shape_extractor.py` — Contour Detection (Tanvi)

This module detects the primary shape boundary. The processing chain is: greyscale conversion $\rightarrow$ Gaussian blur (5×5) $\rightarrow$ Otsu's thresholding (inverted) $\rightarrow$ morphological close/open (elliptical 5×5) $\rightarrow$ `findContours` with `RETR_EXTERNAL`. The largest contour with area $> 0.1\%$ of the image is retained.

Two extraction modes are supported. `Polygon` mode uses `approxPolyDP` returning corner points only (fast, ideal for straight-edged shapes). `Dense` mode retains every k-th contour pixel, preserving smooth curves such as circles (up to 500 points).

7.3 `coord_mapper.py` — Workspace Mapping (Pooja)

A uniform scale factor preserves the input shape's aspect ratio. A 15% inward margin from workspace boundaries is enforced for safety, the shape is centred in the workspace, and the y-axis is inverted (image y increases downward; robot y increases forward).

7.4 `pipeline.py` — Pipeline Orchestrator (Pooja)

Exposes a single `run_pipeline(pdf_path, mode)` function that executes all three CV stages and returns the robot waypoints in metres, the pixel waypoints for the web preview, and the rendered BGR image array.

7.5 `drawing_bot.xml` — MuJoCo Robot Model (Sadhana)

The MJCF XML model includes two actuated revolute joints grounded at $D = 0.075\,\text{m}$ separation, primary links ($L_1 = 0.15\,\text{m}$), secondary links ($L_2 = 0.27\,\text{m}$), a passive pen-tip site, and a marker site for trace visualisation (up to 1 000 spheres).

7.6 `app.py` — Flask Web Backend (Teja)

`app.py` serves a REST API on port 5000. Table 3 lists its four endpoints. Error responses always return JSON for easy frontend parsing.

Table 3: Flask REST API endpoints.

Route	Method	Description
`/`	GET	Serves the HTML frontend.
`/process`	POST	Accepts PDF; runs full pipeline; returns pixel/robot points and base64 preview.
`/simulate`	POST	Launches `sim.py` as a subprocess.
`/status`	GET	Returns processing state and waypoint count.

7.7 `sim.py` — IK Solver and Simulation Driver (Teja)

Calls `run_pipeline()` to obtain ordered robot-space waypoints, then calls `get_ik_radians(x, y)` for each point to compute (θ_L, θ_R). The resulting sequence is streamed to the MuJoCo passive viewer in real time. Trace spheres mark the pen trajectory up to the 1 000-sphere limit.

7.8 Web Frontend — `index.html` (Teja)

Provides drag-and-drop PDF upload, a contour-mode selector (*Polygon* vs. *Dense*), a canvas debug overlay, a live pipeline status panel with four-step progress indicator, and a "Launch Simulation" button calling `/simulate`.

8. Results and Validation

8.1 Web Interface

Figure 2 shows the upload screen; Figure 3 shows the debug overlay and robot-path preview after processing a triangle; Figure 4 shows the pipeline execution log.

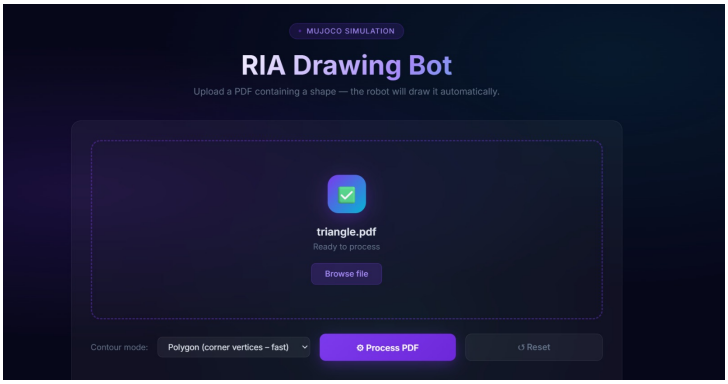

Figure 2: RIA Drawing Bot web interface — PDF upload screen with contour mode selector and "Process PDF" button.

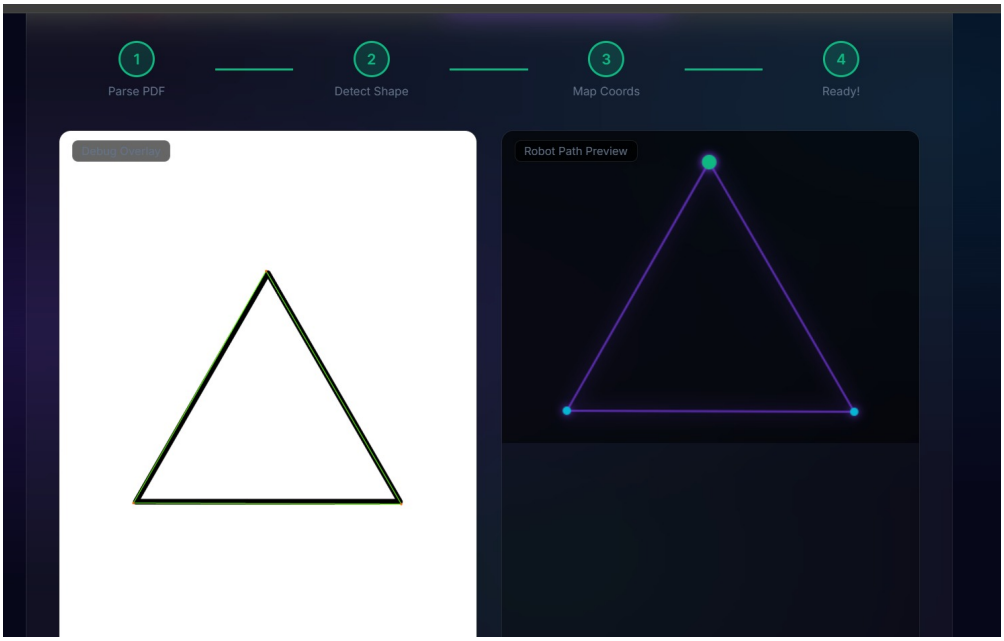

Figure 3: Post-processing view: left panel shows the debug overlay on the uploaded PDF (green contour); right panel shows the robot-path preview mapped to workspace coordinates. All four pipeline stages are marked complete.

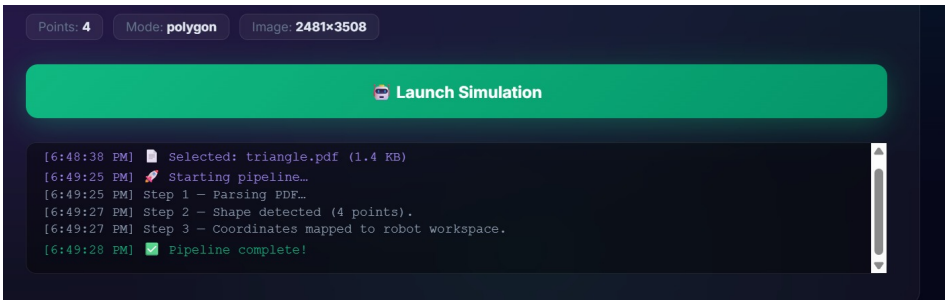

Figure 4: Pipeline execution log confirming all three steps (PDF parse, shape detection, coordinate mapping) and the "Launch Simulation" button.

8.2 Flask Server Startup

Figure 5 shows the Flask development server running from the project directory.

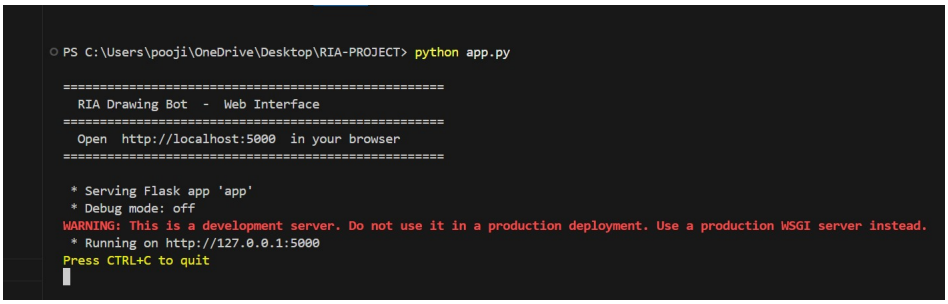

Figure 5: Flask server startup in the VS Code terminal. The server is launched with `python app.py` and is accessible at `http://127.0.0.1:5000`.

8.3 MuJoCo Simulation Outputs

Figures 6 and 7 show the robot drawing a triangle and a cross shape respectively. The green trace marks the pen trajectory. Figures 8 and **??** show the IK live status panel during and after a star drawing, confirming millimetre-level end-effector accuracy.

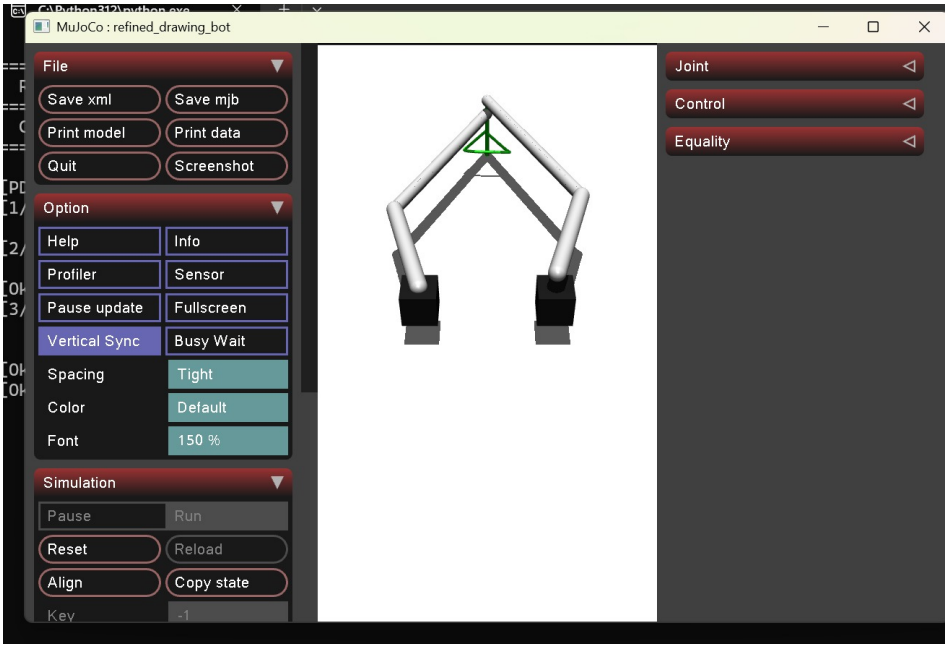

Figure 6: MuJoCo simulation drawing a triangle. The green trace marks the pen path; the robot arm is mid-motion.

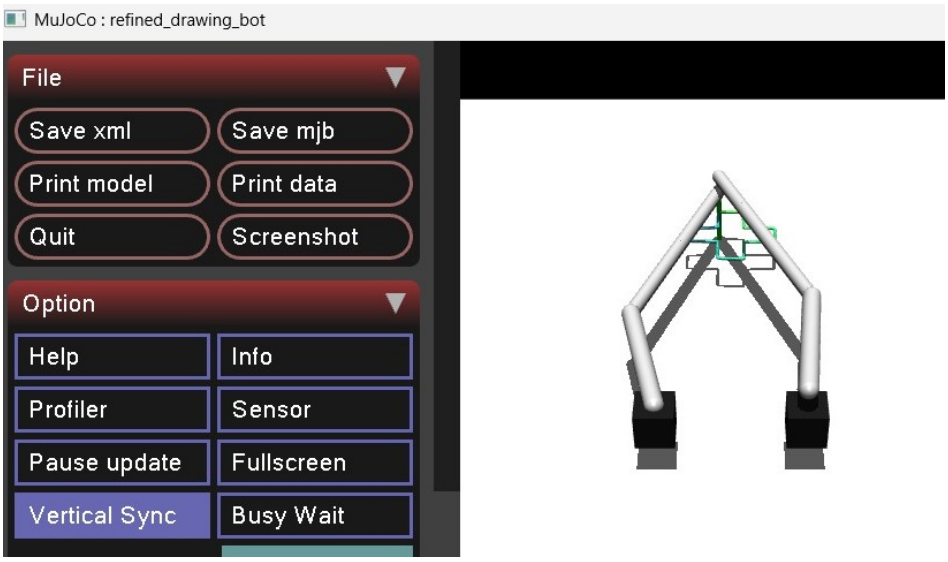

Figure 7: MuJoCo simulation drawing a cross shape (`cross.pdf`). Status panel shows DRAWING state at 99.9% completion with error = 2.557 mm.

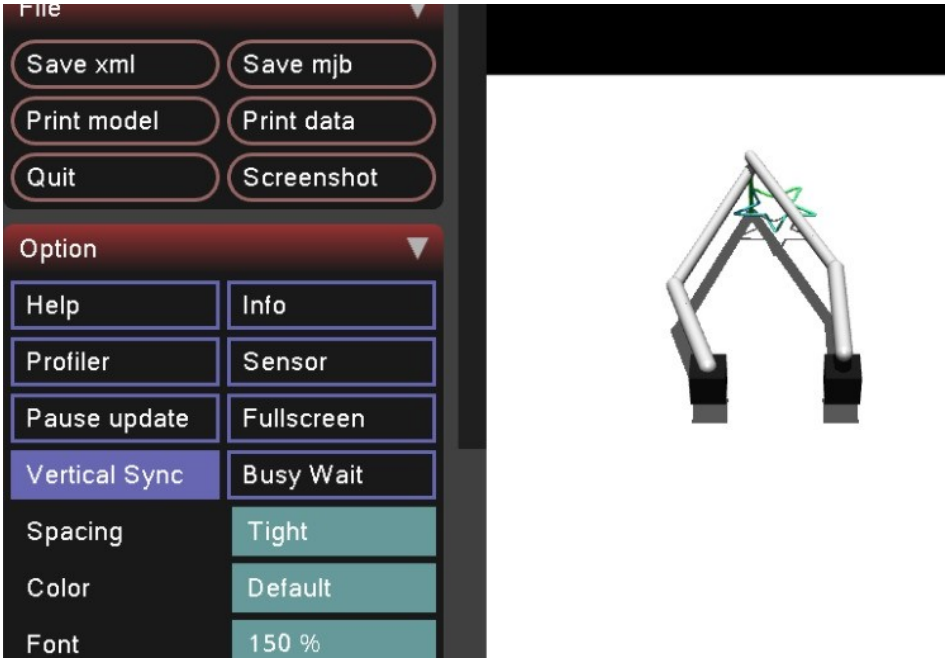

Figure 8: IK live status panel during star drawing. Pen position ($x = -0.0326\,\mathrm{m}$, $y = +0.3975\,\mathrm{m}$), $\theta_L = +16.75$, $\theta_R = -2.49$, error $= 2.572\,\mathrm{mm}$.

Table 4: Pipeline output for representative test shapes.

Shape	**Mode**	**Waypoints**	x **range (m)**	y **range (m)**
Triangle	polygon	4	$[-0.034,\ +0.034]$	$[0.326,\ 0.374]$
Cross	polygon	≈ 8	$[-0.034,\ +0.034]$	$[0.326,\ 0.374]$
Star	polygon	≈ 11	$[-0.034,\ +0.034]$	$[0.326,\ 0.374]$
Circle	dense	≈ 250	$[-0.034,\ +0.034]$	$[0.326,\ 0.374]$

All shapes were successfully scaled to fit within the robot workspace with zero IK singularity violations.

9. Software Stack and Dependencies

Table 5: Software tools and libraries used.

Component	Tool / Library	Role
Physics simulation	MuJoCo	Robot model execution and real-time rendering.
IK / path logic	Python + NumPy	Inverse kinematics, workspace mapping.
CV pipeline	OpenCV	Contour detection, morphological ops.
PDF rendering	PyMuPDF (`fitz`)	PDF-to-image at 300 DPI.
Web backend	Flask, Flask-CORS	REST API, file serving, subprocess control.
Frontend	HTML5, JavaScript	Drag-drop UI, canvas preview.
CAD (planned)	SolidWorks / Fusion 360	Physical linkage design.
Firmware (planned)	FluidNC / C++	ESP32 motor control.

9.1 Planned Hardware

Table 6: Planned physical hardware components.

Component	Part	Purpose
Microcontroller	ESP32 (dual-core, 32-bit)	Motion control, Wi-Fi.
Stepper motors	2× NEMA 17	Primary joint actuation.
Motor drivers	TMC2209 (SilentStepStick)	Micro-stepping.
Pen Z-axis	SG90 micro-servo	Pen lift / drop.
Power (motors)	12 V DC	Stepper power.
Power (logic)	Isolated 3.3 V	ESP32 and driver logic.

10. Real-World Applications

- **Industrial manufacturing**: PCB plotting, laser engraving, and high-speed water-jet cutting.
- **Pick-and-place robotics**: Assembly line sorting — parallel robots are the industry standard for high-speed operations (e.g. ABB FlexPicker).
- **Medical robotics**: Surgical systems requiring sub-millimetre repeatability and high structural rigidity.
- **Automated textiles**: Fabric cutting and leather patterning.

11. Limitations

1. **Non-rectangular workspace**: The five-bar linkage has a heart-shaped reachable area, unlike a standard rectangular CNC grid.

2. **Singularity zones**: Specific arm configurations can cause joints to "lock" if IK angles are not properly clipped.
3. **Simulation only**: Physical hardware has not yet been assembled; the simulation validates IK and pipeline logic only.
4. **Trace limit**: MuJoCo renders up to 1 000 trace spheres; long drawings require chunked rendering.
5. **Open-loop control**: No real-time position feedback — stepper skip errors accumulate silently.

12. Future Scope and Improvements

1. **Physical build**: Assemble ESP32 + NEMA 17 + TMC2209 + SG90 hardware and flash the FluidNC / custom C++ firmware.
2. **Closed-loop feedback**: Add rotary encoders to detect and correct skipped motor steps in real time.
3. **IoT integration**: Replace USB serial with wireless control via the ESP32's onboard Wi-Fi.
4. **Auto-calibration**: Add limit switches for automatic homing and workspace boundary detection.
5. **Multi-tool head**: Swap the pen for a laser diode or a miniature extruder for multi-material capability.
6. **Workspace optimisation**: Explore alternative link-length ratios to maximise the usable rectangular sub-region of the work envelope.

13. Conclusion

This project successfully designed, implemented, and validated in simulation a complete software stack for a five-bar parallel drawing robot. The key contributions are a robust computer-vision pipeline (PyMuPDF + OpenCV) that extracts clean shape contours in two configurable modes; a numerically stable inverse kinematics solver derived from the Law of Cosines; a workspace coordinate mapper that correctly scales, centres, and axis-inverts pixel paths to the robot's physical drawing zone; and a Flask-based web interface enabling non-expert users to upload PDFs, preview extracted shapes, and launch the physics simulation in a single workflow.

The simulation phase confirms the mathematical correctness of the IK model and demonstrates that the parallel kinematic architecture is well-suited for precision 2D drawing applications. Physical hardware construction and closed-loop feedback represent the natural and immediate next steps toward a fully functional drawing robot.

References

[1] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033, 2012.

[2] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[3] Artifex Software, *PyMuPDF Documentation*, https://pymupdf.readthedocs.io, 2024.

[4] A. Ronacher, *Flask Documentation*, https://flask.palletsprojects.com, 2024.

[5] J.-P. Merlet, *Parallel Robots*, 2nd ed. Springer, 2006.

[6] Espressif Systems, *ESP32 Technical Reference Manual*, 2023.